UC San Diego

Large Language Model Overview

CIML Summer Institute 2026

Mai H. Nguyen, Ph.D.

Paul Rodriguez, Ph.D.

SDSC - San Diego Supercomputer Center

LLM Overview Agenda

1:45 - 2:45pm	RAG - Retrieval Augmented Generation
2:45 - 3:00pm	Break
3:00 - 4:00pm	Coding agents
4:00 - 4:30pm	LLM tools and evaluation
4:30 - 4:45pm	Misc topics, Q&A
4:45 - 5:00pm	NAIRR Introduction - Mahidhar Tatineni
5:15 - 5:30pm	Closing remarks

Retrieval Augmented Generation (RAG)

Mai H. Nguyen, Ph.D.

Retrieval Augmented Generation (RAG)

- Definition:
  - Technique to improve capabilities of LLM
- Idea:
  - Improve quality of text generated by LLM by incorporating additional information from an external source
- Approach:
  - Use a *retrieval* component to extract relevant data from an external knowledge base as context to *augment* prompt to help LLM *generate* more accurate and relevant response

Why Use RAG?

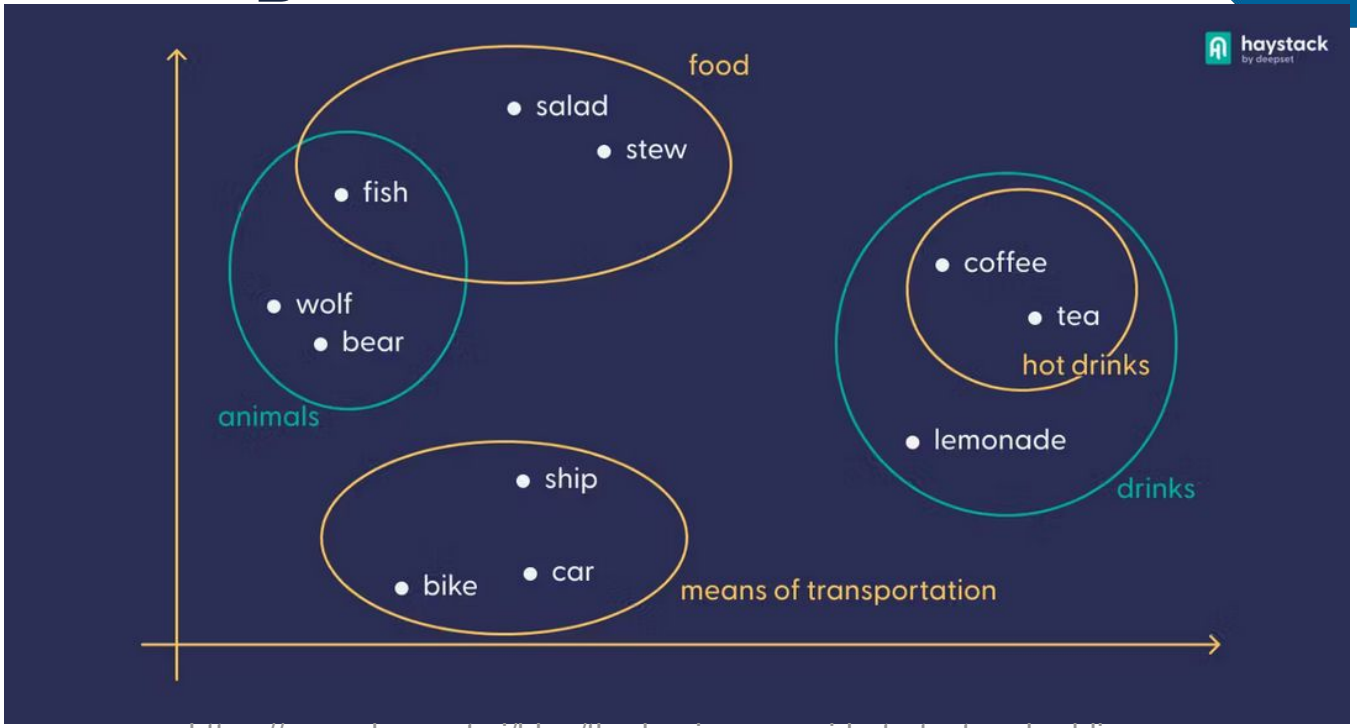
- LLMs
  - Responds to prompts with information from training data
- RAG
  - Allows LLM to access external knowledge base (e.g., company's internal database)
  - Provides most up-to-date and relevant information to LLM in generating response
  - Provides way to validate LLM's response

Embeddings

- Text Embedding:
  - Numeric representation of text (vector of floating point numbers)
  - Capture the semantics of the text
  - Similarity between two embeddings indicates their semantic relatedness (cosine similarity, dot product, etc)

https://cohere.com/blog/text-embeddings

Text Embeddings



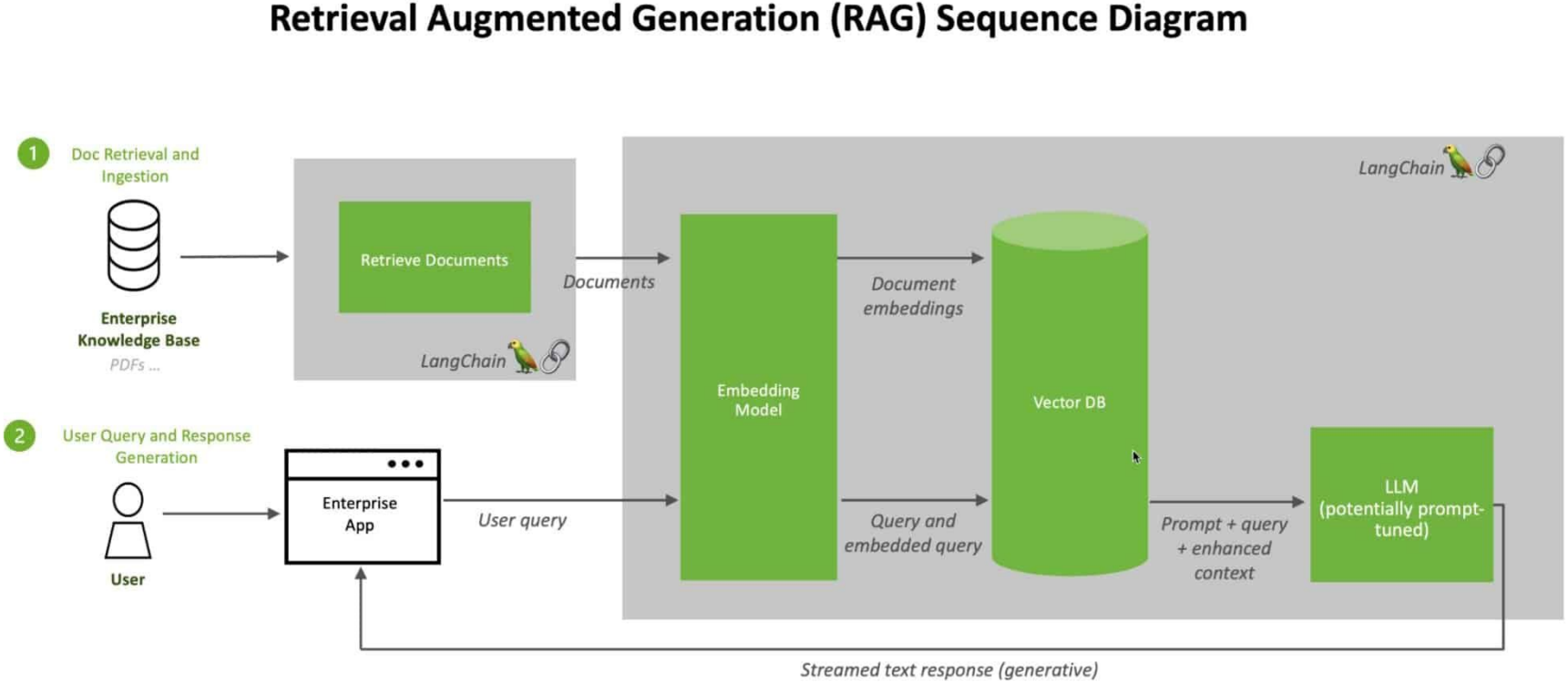
https://www.deepset.ai/blog/the-beginners-guide-to-text-embeddings

Text embeddings allow for search and comparison between user queries and documents in knowledge base

How RAG Works

- User inputs prompt
- Prompt sent to retrieval system
- Retrieval system searches knowledge base and returns top relevant document chunks
- Retrieved chunks are added as context to original prompt
- Augmented prompt sent to LLM

RAG Overview



https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/

RAG Components

- Embedding Model
- Vector Database
- LLM

RAG Components

- **Embedding Model**
  - Document encoding
    - Generates embeddings for documents or text passages in knowledge base
  - Query encoding
    - Generates embeddings for input query
  - Some embedding models
    - HuggingFace: Universal AngIE Embedding, all-MiniLM-L6-v2
    - OpenAI: text-embedding-3-small, text-embedding-3-large (paid)

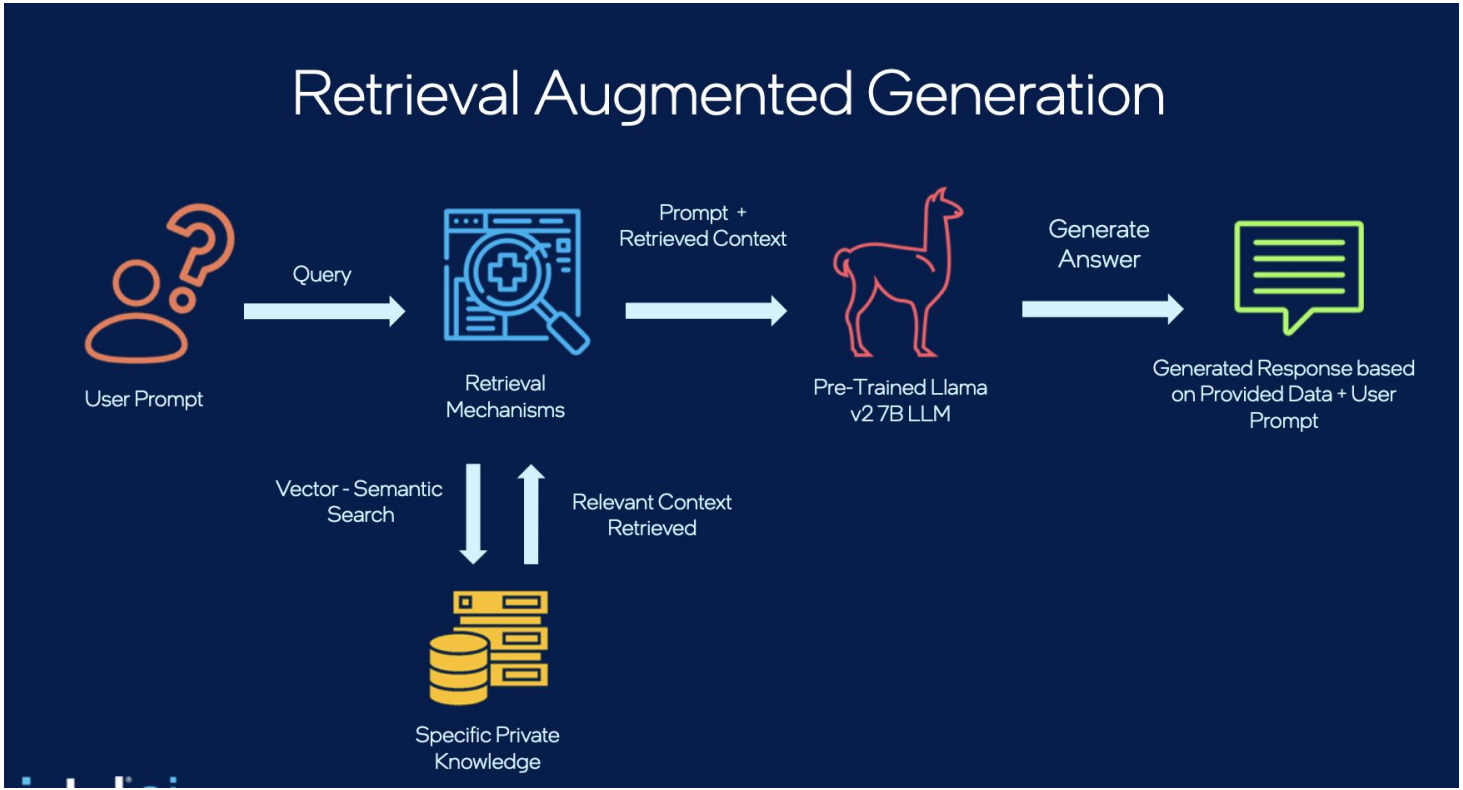
RAG Components

- **Vector Database**
  - Storage
    - Stores precomputed embeddings of documents
  - Similarity search
    - Performs similarity search between query embedding and stored document embeddings to return top-k relevant documents
      - Similarity metrics: cosine similarity, dot product, etc.
    - Optimized for fast and efficient similarity search
  - Some vector databases
    - ChromaDB, Pinecone, Milvus

RAG Components

- **Large Language Model (LLM)**
  - Content generation
    - Takes query augmented with retrieved content
    - Generates response to augmented prompt
  - Some LLMs
    - LLaMa
    - Gemma
    - GPT

RAG Overview



Intel

RAG Hands-On

- RAG Components
  - Embedding model: HuggingFace all-MiniLM-L6-v2
  - Vector database: ChromaDB
  - LLM: Gemma4:e4b
  - LLM server: ollama

RAG Hands-On Outline

- Retrieval Concepts
  - Vectorization to create text embeddings
  - Similarity between embeddings
  - Vector database for storing embeddings
  - Chunking
- Basic RAG
- RAG with LangChain
- Code
  - 4.4_LLM_overview/RAG/RAG_Starter.ipynb

Server Setup on Expanse for RAG

UC San Diego

- Login to Expanse through terminal window
  - Open terminal window on local machine and type in:
    ```
    ssh login.expanse.sdsc.edu -l <username>
    ```
- Pull latest from repo
  - git pull
  - URL: https://github.com/ciml-org/ciml-summer-institute-2026
- In terminal window
  - `jupyter-nairr-gpu-shared-llm`
    - Alias for:
      ```
      galyleo launch --account ${CIML26_ACCOUNT} --reservation
      $CIML26_RES_GPU} --partition nairr-gpu-shared --qos
      ${CIML26_QOS_GPU} --cpus 8 --memory 92 --gpus 1 --time-limit
      04:00:00 --sif ${CIML26_DATA_DIR}/ollama-latest-expanse.sif
      --bind /expanse,/scratch,/cm --nv --quiet
      ```
  - Copy and paste URL in local web browser
- To check queue
  - `squeue -u $USER`

Ollama Setup

UC San Diego

- What is Ollama?
  - Open-source LLM platform that provides access to open-weights LLMs on local system
- In a terminal window in Jupyter Lab, do the following to start an Ollama instance:
  - Set location of Ollama models
    ```
    export OLLAMA_MODELS=/cm/shared/examples/sdsc/ciml/2026/ollama_models/
    ```
  - Find randomized port for Ollama server to run on; needed on Expanse to avoid collisions when multiple users run instances on the same host.
    - ```
      export OLLAMA_PORT="$(shuf -i 3000-65000 -n 1)" && echo "${OLLAMA_PORT}" > ~/.ollama_port
      ```
    - ```
      export OLLAMA_HOST="127.0.0.1:${OLLAMA_PORT}"
      ```
  - Start Ollama server as a background job, and set up log file
    - ```
      ollama serve > ollama.log 2>&1 &
      ```
  - See what ollama models are available
    - ```
      ollama list
      ```  # Should see gemma4:e4b and qwen3:14b

Ollama Shutdown - When Done

- In terminal window (in Jupyter Lab) where you started Ollama
  - Bring the Ollama job to the foreground
    - `jobs` # Should see : [1]+ Running ollama serve > ollama.log 2>&1 &
    - `fg`
  - Stop the Ollama server
    - `<Cntrl>-C`
  - Check that nothing is running in the background
    - `jobs` # Should see nothing

Ollama Setup on Local System

- To pull Ollama model to your local system:
  - Unset the Ollama models location
    - `unset $OLLAMA_MODELS`
    - `echo $OLLAMA_MODELS  # Should see blank`
  - Pull the model
    - `ollama pull <model>`
  - To see available models
    - https://ollama.com/library

Resources

- RAG
  - https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag
  - Step-by-Step Tutorial on Integrating Retrieval-Augmented Generation (RAG) with Large Language Models | by Novita AI | Apr, 2024 | Medium
    ollama/examples/langchain-python-rag-document/main.py at main
- Embedding model
  - https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- ChromaDB
  - https://docs.trychroma.com/
  - https://colab.research.google.com/drive/181Kummxd8yOyRqFu8I0aqjs2aqnOy4Fu?usp=sharing

Resources

- LangChain
  - https://python.langchain.com/v0.1/docs/use_cases/question_answering/quickstart/
  - https://api.python.langchain.com/en/latest/chains/langchain.chains.retrieval_qa.base.RetrievalQA.html
  - https://python.langchain.com/v0.2/docs/integrations/vectorstores/chroma/
  - https://python.langchain.com/v0.2/docs/integrations/text_embedding/huggingfacehub/
- Ollama:
  - https://www.ollama.com/
  - Ollama | 🦜🔗 LangChain